DBMS – ADVANCED SQL QUERY SYSTEM

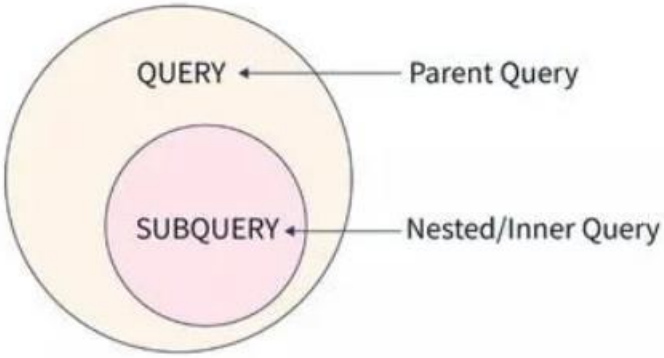
TATA CLiQ E-Commerce Order Management System

OBJECTIVES

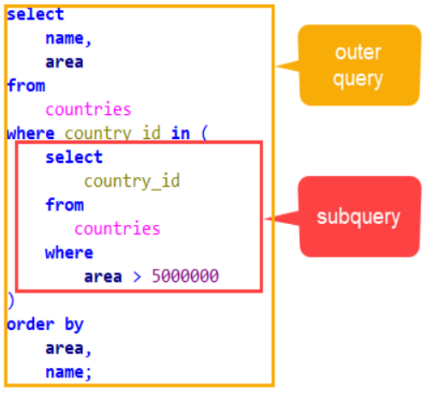
1. To implement subqueries and nested queries.
2. To find products above and below average price.
3. To identify customers with maximum purchases.
4. To generate customer and product sales reports.
5. To analyse category-wise sales.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query. The inner query executes first, and its result is used by the outer query.



Basic Structure



Types of Subqueries

Subquery	Description
Single-Row Subquery	Returns one row of results that consists of one column to the outer query
Multiple-Row Subquery	Returns more than one row of results to the outer query
Multiple-Column Subquery	Returns more than one column of results to the outer query
Correlated Subquery	References a column in the outer query. Executes the subquery once for every row in the outer query
Uncorrelated Subquery	Executes the subquery first and passes the value to the outer query

1.1 Find Orders Greater Than Average Order Amount

```
SQL> SELECT *
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT >
  4  (
  5      SELECT AVG(TOTAL_AMOUNT)
  6      FROM ORDERS
  7  );

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ -------------
      1002         102 25-SEP-26         3000 Shipped
      1003         103 22-SEP-26         3200 Pending
      1005         104 24-SEP-26         4200
```

2. Find Products Above Average Price

This query identifies products whose price is greater than the average price of all products.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM PRODUCT
  3  WHERE PRICE >
  4  (
  5      SELECT AVG(PRICE)
  6      FROM PRODUCT
  7  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------
     PRICE
----------
       101
Samsung Galaxy S24
     69999

       102
iPhone 15
     64999
```

2.1 Products Below Average Price

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM PRODUCT
  3  WHERE PRICE <
  4  (
  5      SELECT AVG(PRICE)
  6      FROM PRODUCT
  7  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
----
     PRICE
----------
       104
Nike Running Shoes
      3999

       105
Lakme Face Cream
       899
```

3. Identify Customers with Maximum Purchases

This query identifies the customer who has made the highest total purchase.

Calculate Customer-wise Purchase:

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM ORDERS
  4  GROUP BY CUSTOMER_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101           4200
        102           3000
        103           3200
        104           4200
```

Find the Maximum Purchase:

```
SQL> SELECT MAX(TOTAL_PURCHASE)
  2  FROM
  3  (
  4      SELECT CUSTOMER_ID,
  5             SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  6      FROM ORDERS
  7      GROUP BY CUSTOMER_ID
  8  );

MAX(TOTAL_PURCHASE)
-------------------
               4200
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  5  FROM CUSTOMER C
  6  JOIN ORDERS O
  7  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
  9  HAVING SUM(O.TOTAL_AMOUNT) =
 10  (
 11      SELECT MAX(TOTAL_PURCHASE)
 12      FROM
 13      (
 14          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 15          FROM ORDERS
 16          GROUP BY CUSTOMER_ID
 17      )
 18  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
        101 Thansila
Begam                                                        4200

        104 Divya
Rani                                                         4200
```

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

Purpose:

This report identifies customers who placed orders whose value is greater than the overall average order value.

```
SQL> SELECT DISTINCT
  2         C.CUSTOMER_ID,
  3         C.FIRST_NAME,
  4         C.LAST_NAME,
  5         O.ORDER_ID,
  6         O.TOTAL_AMOUNT
  7  FROM CUSTOMER C
  8  JOIN ORDERS O
  9  ON C.CUSTOMER_ID = O.CUSTOMER_ID
 10  WHERE O.TOTAL_AMOUNT >
 11  (
 12      SELECT AVG(TOTAL_AMOUNT)
 13      FROM ORDERS
 14  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
        102 Priya
Devi                                                     1002         3000

        103 Rahul
Kumar                                                    1003         3200

        104 Divya
Rani                                                     1005         4200
```

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated using ORDER_ITEMS.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OI.QUANTITY * P.PRICE) AS TOTAL_SALES
  4  FROM PRODUCT P
  5  JOIN ORDERITEM OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  HAVING SUM(OI.QUANTITY * P.PRICE) >
  9  (
 10      SELECT AVG(PRODUCT_SALES)
 11      FROM
 12      (
 13          SELECT SUM(OI.QUANTITY * P.PRICE) AS PRODUCT_SALES
 14          FROM PRODUCT P
 15          JOIN ORDERITEM OI
 16          ON P.PRODUCT_ID = OI.PRODUCT_ID
 17          GROUP BY P.PRODUCT_ID
 18      )
 19  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
----
TOTAL_SALES
-----------
       101
Samsung Galaxy S24
     139998
```

6. Customer Purchase Report

A detailed customer purchase report combines CUSTOMER and ORDERS tables.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  5         AVG(O.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  6  FROM CUSTOMER C
  7  JOIN ORDERS O
  8  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
 10  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
----
NUMBER_OF_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
---------------- -------------- -------------------
        101
Thansila Begam
               2           4200                2100

        104
Divya Rani
               1           4200                4200
```

7. Advanced Product Sales Report

This report combines PRODUCT and ORDER_ITEMS tables.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         COUNT(DISTINCT OD.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  5         SUM(OD.QUANTITY * P.PRICE) AS TOTAL_SALES,
  6         AVG(P.PRICE) AS AVERAGE_SELLING_PRICE
  7  FROM PRODUCT P
  8  JOIN ORDER_DETAILS OD
  9  ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
----
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- ------------------- ----------- --------------------
       201
Laptop
               1                   3      150000                50000

       203
Mobile
               1                   2       40000                20000
```

8. Category-wise Advanced Sales Report

Category-wise sales analysis helps identify which product categories generate higher revenue.

```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         COUNT(DISTINCT P.PRODUCT_ID) AS NUMBER_OF_PRODUCTS,
  4         NVL(SUM(OD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD,
  5         NVL(SUM(OD.QUANTITY * P.PRICE), 0) AS TOTAL_SALES,
  6         AVG(P.PRICE) AS AVERAGE_PRODUCT_PRICE
  7  FROM CATEGORY C
  8  JOIN PRODUCT P
  9  ON C.CATEGORY_ID = P.CATEGORY_ID
 10  LEFT JOIN ORDER_DETAILS OD
 11  ON P.PRODUCT_ID = OD.PRODUCT_ID
 12  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 13  ORDER BY TOTAL_SALES DESC;

CATEGORY_ID CATEGORY_NAME
----------- --------------------------------------------------
NUMBER_OF_PRODUCTS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRODUCT_PRICE
------------------ ------------------- ----------- ---------------------
          1 Electronics
                 3                   3      150000            35999.6667

          3 Laptops
                 2                   2       40000               37499.5

          6 Beauty
                 2                   2        4000                1449.5
```